

The Happy Fly

A beginner learning guide for Python in VS Code | Version 2



Pavle Novakovic

Flies Game | Learn by experimenting

What you will build

A fly circles a light-gray window 7 times, lands on a brown target, and displays a happy message. This is an automatic animation: your first exercise in variables, drawing, movement and program states.

A small, practical learning path

1. Run one line

Learn where to type Python and where to type terminal commands.

2. Open a window

Understand setup, events and the repeating main loop.

3. Draw the scene

Use coordinates and RGB colours to place simple shapes.

4. Animate the fly

Track an angle, count circles, then change state.

5. Change one thing

Predict the result, edit a value, run again and compare.

The complete flies_V2.py source is on pages 5-7 for you to type yourself. Do not type the line numbers; preserve indentation. The landing message includes YUMMY! No image or sound files are needed.

01 / Set up and run

Windows-focused instructions | Keep this in its own flies_game folder

1. Open your practice folder

In VS Code, choose File > Open Folder and select a folder named flies_game. Create a blank flies_V2.py in that folder and type the code from pages 5-7 into it. Install the Python extension published by Microsoft if it is not already installed.

2. Select Python

Press Ctrl+Shift+P, choose Python: Select Interpreter, and select your installed Python. Open Terminal > New Terminal. An interpreter is the program that executes your Python code.

3. Install the drawing library

Type these commands in the terminal, one at a time. Do not type them at a Python >>> prompt.

```
python --version  
python -m pip install --only-binary=:all: pygame-ce
```

If python is not recognised on Windows, try py instead in both commands. If neither works, install Python from python.org/downloads, then reopen VS Code. The package is named pygame-ce, but the code still uses import pygame. If you have already installed it successfully, skip this step.

4. Run the animation

```
python flies_V2.py
```

Alternatively, open flies_V2.py and use Run Python File in Terminal. The fly completes 7 circles in about 15 seconds, then lands. Press R to restart; Esc or the window close button exits.

If pygame cannot be found

The installer and VS Code may be using different Python interpreters. Select the interpreter again, open a fresh terminal, and install using that same Python. Using python -m pip ties pip to that terminal interpreter.

02 / Understand the building blocks

Colour labels describe responsibilities; they are not Python syntax

1 SETUP

```
pygame.init(); display.set_mode(); Clock()
```

Initialise Pygame, open the window and create a clock. Font is created once and reused.

2 DATA

```
radius; angle; state; completed_circles
```

Variables hold values that the program reads or changes. A tuple such as (40, 40, 40) groups RGB colour values.

3 INPUT

```
event.get(); QUIT; KEYDOWN; tick(60)
```

Read window and keyboard events. tick limits the frame rate and returns elapsed milliseconds; divide by 1000 for seconds.

4 UPDATE

```
if / elif; cos(); sin(); hypot()
```

Choose the active state and update position. Only one state branch runs per frame.

5 DRAW

```
fill(); draw.circle(); blit(); flip()
```

Clear the previous frame, draw shapes and text, then show the completed image.

6 EXIT

```
pygame.quit()
```

Release Pygame resources after the main loop ends.

03 / Learn by experimenting

Try a prediction before running each change

First: practise at the >>> prompt

Type python in the terminal to enter the interactive Python prompt. Then enter these lines individually. Type exit() when finished, before running terminal commands again.

```
print("Hello Fly")
radius = 120
print(radius * 2)
exit()
```

How the movement works

The top-left corner of the window is (0, 0). x increases to the right; y increases downward. cos(angle) and sin(angle) create offsets around the centre. One full turn is math.tau radians (2 times pi).

The program keeps the total angle instead of resetting it each lap. Dividing it by math.tau and taking int counts finished laps. After 7 laps, the state changes from flying to landing, then to happy.

dt is the time since the previous frame. Multiplying speed by dt makes movement depend on elapsed time. During landing, hypot measures the distance to the target; a final snap prevents overshooting.

Three small exercises

1. Change loops_required from 7 to 3. Predict when landing starts.
2. Change radius from 120 to 180. Observe the wider orbit.
3. Change angular_speed from 3.0 to 1.5. Predict the lap time.

Read errors as clues

IndentationError: check spaces, especially inside while, if and elif blocks. NameError: check spelling and capitals. A frozen window: ensure event handling remains inside the main loop. Save the file before rerunning.

Official references

code.visualstudio.com/docs/python/run

code.visualstudio.com/docs/languages/python

www.pygame.org/docs/

www.python.org/downloads/

04 / Complete program

flies_V2.py | Part 1 of 3 | Continue across pages; omit line numbers

```
1  """Happy Fly: Circle 7 times, land, then celebrate."""
2
3  import math
4  import pygame
5
6  # [1 SETUP] Create the window and reusable tools.
7  pygame.init()
8
9  WIDTH, HEIGHT = 900, 700
10 screen = pygame.display.set_mode((WIDTH, HEIGHT))
11 pygame.display.set_caption("Happy Fly V2")
12
13 clock = pygame.time.Clock()
14 font = pygame.font.Font(None, 36)
15
16 LIGHT_GRAY = (180, 180, 180)
17 BROWN = (130, 85, 45)
18 WHITE = (255, 255, 255)
19 BLACK = (10, 10, 10)
20 GREEN = (20, 100, 40)
21
22 # [2 DATA] Position, speed and animation state.
23 center_x, center_y = WIDTH / 2, HEIGHT / 2
24
25 radius = 120
26 loops_required = 7
27 angular_speed = 3.0      # Radians per second.
28 landing_speed = 140.0    # Pixels per second.
29
30 target_x, target_y = center_x, center_y + 150
31
32 angle = 0.0
33 completed_circles = 0
34 fly_x, fly_y = center_x + radius, center_y
35
36 state = "flying"
37 running = True
38
```

05 / Complete program

flies_V2.py | Part 2 of 3 | Continue across pages; omit line numbers

```
39 while running:
40     # [3 INPUT] Measure time and process window events.
41     dt = clock.tick(60) / 1000.0
42
43     for event in pygame.event.get():
44         if event.type == pygame.QUIT:
45             running = False
46
47         elif event.type == pygame.KEYDOWN:
48             if event.key == pygame.K_ESCAPE:
49                 running = False
50
51             elif event.key == pygame.K_r:
52                 angle = 0.0
53                 completed_circles = 0
54                 fly_x, fly_y = center_x + radius, center_y
55                 state = "flying"
56
57     if not running:
58         break
59
60     # [4 UPDATE] Fly in circles, then move to the target.
61     if state == "flying":
62         angle = min(
63             angle + angular_speed * dt,
64             loops_required * math.tau
65         )
66
67         completed_circles = min(
68             int(angle / math.tau),
69             loops_required
70         )
71
72         fly_x = center_x + radius * math.cos(angle)
73         fly_y = center_y + radius * math.sin(angle)
74
75         if angle >= loops_required * math.tau:
76             state = "landing"
77
78     elif state == "landing":
79         dx = target_x - fly_x
80         dy = target_y - fly_y
81
82         distance = math.hypot(dx, dy)
83         step = landing_speed * dt
84
85         if distance <= step:
86             fly_x, fly_y = target_x, target_y
87             state = "happy"
88         else:
89             fly_x += dx / distance * step
90             fly_y += dy / distance * step
91
```

06 / Complete program

flies_V2.py | Part 3 of 3 | Continue across pages; omit line numbers

```
92     # [5 DRAW] Paint a complete fresh frame.
93     screen.fill(LIGHT_GRAY)
94
95     pygame.draw.circle(
96         screen, BROWN,
97         (int(target_x), int(target_y)), 30
98     )
99
100    pygame.draw.rect(
101        screen, BROWN,
102        (int(target_x) - 40, int(target_y) + 10, 80, 15)
103    )
104
105    x, y = round(fly_x), round(fly_y)
106
107    pygame.draw.circle(screen, WHITE, (x - 7, y - 8), 7)
108    pygame.draw.circle(screen, WHITE, (x + 7, y - 8), 7)
109    pygame.draw.circle(screen, BLACK, (x, y), 10)
110
111    label = f"Circles: {completed_circles}/{loops_required}"
112    screen.blit(font.render(label, True, BLACK), (25, 20))
113
114    if state == "happy":
115        message = font.render(
116            "The Fly is Happy! BZZZ! YUMMY!",
117            True, GREEN
118        )
119
120        screen.blit(
121            message,
122            message.get_rect(center=(WIDTH // 2, 80))
123        )
124
125    hint = font.render("R: restart    Esc: quit", True, BLACK)
126    screen.blit(hint, (25, HEIGHT - 40))
127
128    pygame.display.flip()
129
130 # [6 EXIT] Release the window when the loop finishes.
131 pygame.quit()
```